Universidade de Brasília
Instituto de Ciências Exatas
Departamento de Ciência da Computação

Redocumentação de Sistemas Legados Java Usando uma Arquitetura de Multiagentes

Murillo Edson de Carvalho Souza

Versão revisada após a banca de qualificação
Mestrado em Informática
Orientador Prof. Dr. Rodrigo Bonifácio de Almeida

Brasília
2026

Resumo

Esta versão mantém o conteúdo do projeto apresentado à qualificação e incorpora apenas alterações diretamente relacionadas às recomendações da banca. O escopo empírico foi delimitado à geração e à avaliação de JavaDocs por modelos de linguagem em um sistema Java real. O texto reforça a caracterização do SARSB, formaliza o método, explicita a arquitetura multiagente e a autoria compartilhada do pipeline, estabelece uma linha de base documental e cria rastreabilidade entre os achados do Estudo 01 e o planejamento do Estudo 02.

Palavras-chave: redocumentação; sistemas legados; JavaDoc; modelos de linguagem; arquitetura multiagente; avaliação empírica.

Capítulo 1

Introdução

Introdução A evolução de software é inerentemente contínua e impulsionada por mudanças ambientais, organizacionais e tecnológicas, tornando a manutenção e a compreensão de sistemas legados atividades progressivamente mais complexas ao longo do tempo [1]. Grande parte do custo de manutenção de software está associada à compreensão de programas existentes, à recuperação de conhecimento arquitetural e à interpretação de regras de negócio implícitas no código [2]. Sistemas legados frequentemente apresentam degradação arquitetural, conhecimento tácito disperso e dependências complexas, aumentando significativamente a carga cognitiva necessária para uma manutenção segura [3]. A ausência de documentação atualizada e o enfraquecimento da memória organizacional tornam tarefas de rastreamento arquitetural e reconstrução de intenção de design particularmente difíceis em sistemas corporativos [4]. Esse cenário é agravado quando as equipes responsáveis pela evolução e manutenção do sistema não correspondem às equipes originalmente envolvidas em sua concepção. A rotatividade organizacional, terceirização, fragmentação de responsabilidades e knowledge vaporization contribuem para a perda de conhecimento tácito e para o aumento da assimetria informacional entre os membros das equipes[5]. A documentação técnica atualizada torna-se um ativo essencial para garantir continuidade operacional, rastreabilidade e transparência institucional [4]. Contudo, pesquisas indicam que grande parte dos sistemas legados carece de documentação técnica atualizada e consistente, fazendo com que conhecimento crítico permaneça restrito a especialistas do domínio, o que dificulta atividades de manutenção e eleva o risco de regressões e inconsistências durante a evolução do software [6]. Esse cenário se agrava em arquiteturas complexas, que envolvem diferentes bibliotecas e tecnologias (por exemplo, em sistemas Java multicamadas). Em tais arquiteturas, o comportamento de um único método pode envolver interações complexas entre módulos de configuração, serviços, entidades de domínio, repositórios e interfaces REST. Estudos

sobre processos de documentação indicam que documentação ausente, desatualizada ou inconsistente compromete atividades de manutenção e aumenta a ocorrência de defeitos ao longo do ciclo de desenvolvimento de software [7]. Diversos estudos têm demonstrado que lacunas documentais, incluindo ausência, incompletude e desatualização de artefatos técnicos, figuram entre as principais origens de technical debt em projetos de software. A literatura sobre Technical Debt aponta que problemas relacionados à documentação podem comprometer manutenibilidade e evolvabilidade do software, especialmente em sistemas corporativos complexos [8]. Em contextos de elevada pressão evolutiva,

equipes de manutenção frequentemente adotam soluções paliativas e arquiteturalmente inadequadas, incluindo duplicações acidentais de código, acoplamentos indevidos e adaptações desconectadas de padrões arquiteturais, aumentando significativamente a complexidade estrutural e o custo de evolução do software ao longo do tempo [9]. Estudos empíricos em ambientes industriais demonstram que a ausência de documentação estruturada sobre fluxos internos, relacionamentos arquiteturais e decisões de design compromete atividades de compreensão de código, impact analysis e manutenção evolutiva, aumentando o risco de inconsistências em modificações subsequentes [10]. Assim, lacunas documentais não representam apenas um déficit informacional, mas se consolidam como um vetor estruturante da dívida técnica, com efeitos acumulativos sobre qualidade, risco e eficiência operacional. Nesse cenário, reconstruir documentação técnica com qualidadesl, auditável e semanticamente consistente tornou-se um desafio estratégico da engenharia de software contemporânea. Embora pesquisas recentes avancem no uso de mineração de repositórios, mecanismos automatizados de rastreabilidade e Large Language Models aplicados à sumarização e geração de documentação, parte significativa da literatura ainda apresenta limitações relacionadas ao uso de datasets públicos, ambientes experimentais artificialmente controlados e ausência de validação em sistemas legados reais mantidos em contexto industrial [11]. Apesar dos avanços recentes em técnicas de code summarization e geração automatizada de documentação baseada em modelos de linguagem, a literatura ainda aponta limitações importantes relacionadas à ausência de validação robusta em sistemas legados reais, especialmente em ambientes industriais caracterizados por elevado acoplamento arquitetural, heterogeneidade tecnológica e alta complexidade operacional [12]. Por isso, a reconstrução documental permanece como um tema de fronteira: demanda metodologias que conciliem automação, precisão semântica e auditabilidade, ao mesmo tempo em que exigem estudos empíricos que reflitam a realidade de sistemas corporativos, seus fluxos heterogêneos e sua dinâmica histórica de evolução. Diante da crescente complexidade associada à manutenção de sistemas legados, abordagens baseadas em automação, engenharia cognitiva e inteligência artificial vêm sendo

exploradas como mecanismos de apoio à compreensão de software. Nesse contexto, Large LanguageModels(LLMs)têmdemonstradoelevadopotencialemtarefascomosourcecode summarization, explicação contextual de métodos, recuperação de conhecimento arquitetural e geração automática de documentação técnica [13]. Apesar dos avanços recentes, parte significativa das avaliações experimentais envolvendo LLMs aplicados à engenharia de software ainda se concentra em benchmarks sintéticos ou repositórios open-source previamente filtrados. Esse cenário limita a compreensão sobre o desempenho efetivo dessas abordagens em sistemas reais, marcados por alta complexidade arquitetural, dívida técnica, heterogeneidade tecnológica e manutenção institucional contínua [14]. Estudos recentes indicam que abordagens baseadas em LLMs têm apresentado resultados promissores em tarefas relacionadas à sumarização de código, geração automática de mensagens de commit e apoio à compreensão e

manutenção de software [15]. Assim, a literatura ainda carece de investigações robustas e empiricamente validadas que examinem o uso de LLMs para sumarização e documentação de código em sistemas reais, revelando uma lacuna importante para pesquisas futuras.

1.1 Contexto específico e linha de base do SARSB

O Sistema de Acompanhamento da Regulação do Saneamento Básico SARSB é mantido pela Agência Nacional de Águas e Saneamento Básico e apoia atividades regulatórias do setor de saneamento. O backend analisado foi desenvolvido em Java e organizado em camadas de configuração, modelo, serviços, persistência, mapeamento, utilitários, APIs, relatórios e agendamentos. A versão examinada contém 524 arquivos Java e aproximadamente 25 mil linhas de código. O inventário estrutural reproduzido pelo pipeline identificou 1.478 alvos públicos: 521 classes, 866 métodos, 84 construtores e 7 enumerações.

Antes da intervenção, somente 9 dos 1.478 alvos inventariados possuíam JavaDoc, correspondendo a 0,61%. Os outros 1.469 alvos, ou 99,39%, não apresentavam JavaDoc. Essa medida constitui a linha de base documental do estudo e delimita a lacuna efetivamente tratada pelo pipeline. Os números referem-se aos elementos públicos identificados pelo JavaParser na versão analisada, e não devem ser generalizados para outras versões do sistema.

Uma prova de conceito anterior com Dynatrace registrou endpoints com parâmetros indefinidos, falhas de autenticação e uso elevado de CPU por requisição. Esses achados são apresentados apenas como evidências contextuais de dificuldades operacionais; não demonstram relação causal entre ausência de JavaDoc e desempenho. A motivação empírica desta dissertação decorre da combinação entre baixa cobertura documental, transferência de conhecimento entre equipes e necessidade institucional de manutenção rastreável.

1.2 Problema de pesquisa

O problema investigado é a baixa disponibilidade de documentação de código útil e confiável para apoiar a compreensão e a manutenção do SARSB. Neste trabalho, documentação técnica designa especificamente JavaDocs associados a classes, métodos, construtores e enumerações Java. Documentação de APIs, arquitetura e testes manuais integra o projeto tecnológico mais amplo, mas não compõe o núcleo empírico desta dissertação.

Questão principal: como desenvolvedores percebem a qualidade, a utilidade, a confiabilidade e o potencial de adoção de JavaDocs gerados por modelos de linguagem durante a revisão de código de um sistema Java multicamadas real?

1.3 Objetivos

O objetivo geral é avaliar empiricamente JavaDocs gerados por modelos de linguagem em um sistema Java real, considerando a percepção de desenvolvedores e evidências técnicas do processo de geração e revisão.

- caracterizar o sistema e estabelecer a cobertura documental anterior à intervenção;
- descrever de forma reproduzível o pipeline e os papéis dos agentes utilizados para gerar, validar e integrar JavaDocs;
- identificar em quais componentes os JavaDocs acrescentam valor semântico e quais problemas afetam correção, consistência, linguagem e sustentabilidade;
- analisar como seleção de alvos, revisão humana e estratégia de entrega influenciam confiança e adoção;
- usar os achados do Estudo 01 para planejar um Estudo 02 comparativo e rastreável.

1.4 Contribuições e delimitação de autoria

A contribuição central da dissertação é a avaliação empírica e contextual de JavaDocs gerados por modelos de linguagem em um sistema público real. O pipeline foi desenvolvido no âmbito de um projeto colaborativo e contém contribuições de outros participantes; portanto, sua implementação não é reivindicada como contribuição autoral exclusiva. A dissertação contribui ao caracterizar o caso, formalizar o protocolo de avaliação, sistematizar categorias de qualidade e converter os achados do estudo preliminar em requisitos verificáveis para o refinamento da solução.

Capítulo 2

Fundamentação teórica e trabalhos relacionados

Fundamentação Teórica A manutenção de software e, em particular, a compreensão de código-fonte em sistemas legados e de grande porte, tem sido apontada como uma das atividades mais custosas e intensivas em esforço cognitivo na engenharia de software. A literatura clássica sobre manutenção e evolução de software reconhece que atividades de compreensão de programas consomem parcela substancial do esforço de manutenção, especialmente em sistemas legados marcados por documentação escassa, desatualizada ou semanticamente inconsistente [21].Em ambientes corporativos e governamentais, a ausência de mecanismos eficazes de documentação e compreensão arquitetural compromete diretamente atributos críticos como qualidade, confiabilidade e continuidade operacional dos sistemas. Em aplicações de larga escala e continuamente evolutivas, marcadas por elevado acoplamento estrutural, intensa atividade de manutenção e operação contínua em produção, compreender rapidamente o comportamento do software torna-se um desafio cognitivo significativo. Sob essa ótica, soluções capazes de apoiar desenvolvedores na construção rápida de modelos mentais consistentes sobre fluxos internos, dependências e efeitos das modificações realizadas tornam-se essenciais para reduzir riscos operacionais e aumentar a eficiência de manutenção [22]. É nesse cenário que surgem as técnicas desource code summarizatione, mais recentemente, o uso de Modelos de Linguagem de Grande Escala (LLMs) para geração automática de documentação. Esta seção apresenta o pano de fundo teórico da pesquisa, organizada em três eixos principais: (i)source code summarization, da literatura clássica às abordagens atuais; (ii) LLMs e ferramentas específicas para compreensão e documentação de código, com foco em foundation models; e (iii) estratégias de validação das sumarizações geradas, com ênfase no uso de juízes humanos e automáticos.

2.1 Source Code Summarization

Source code summarizationpode ser compreendido como o processo de geração automática de descrições em linguagem natural capazes de representar, de forma sintética e semanticamente coerente, o comportamento, o propósito e os elementos relevantes de componentes de software, como classes e métodos. Essas abordagens têm sido utilizadas como mecanismos de apoio àcompreensão de programas, especialmente em sistemas legados caracterizados por documentação escassa ou desatualizada, buscando reduzir o esforço necessário à compreensão do código e apoiar atividades de manutenção, evolução e transferência de conhecimento entre desenvolvedores [12]. Em sistemas com longa história evolutiva, a documentação frequentemente não acompanha a evolução do software, tornando-se desatualizada em

relação ao código-fonte [20]. Técnicas de sumarização automática vêm sendo investigadas como mecanismos de apoio à documentação de software, permitindo gerar descrições em linguagem natural a partir de artefatos do sistema e apoiando atividades de compreensão e manutenção de software [23] Historicamente, as primeiras abordagens desource code summarizationforam predominantemente baseadas em regras, heurísticas e templates construídos a partir da estrutura sintática do código-fonte. Essas técnicas utilizavam informações como nomes de métodos, parâmetros e comentários para produzir descrições em linguagem natural, contribuindo para os primeiros avanços da área. Entretanto, apresentavam limitações na capacidade de capturar aspectos semânticos mais profundos do software, especialmente em cenários complexos de compreensão e manutenção [24]. Posteriormente, surgiram abordagens baseadas em recuperação de informação (Information Retrieval– IR) e modelos estatísticos, que exploravam correlações entre elementos presentes no código-fonte e descrições textuais obtidas a partir de grandes coleções de software. Embora tenham ampliado a capacidade de generalização das técnicas anteriores, essas abordagens ainda apresentavam dificuldades para representar adequadamente relações estruturais, dependências e contexto semântico, comprometendo a qualidade e a legibilidade das sumarizações geradas [25]. Mais recentemente, arquiteturas baseadas emTransformers, como CodeBERT, CodeT5 e, posteriormente, osFoundation Modelsbaseados em Large Language Models (LLMs), passaram a representar o estado da arte em tarefas de compreensão e sumarização de código, explorando representações semânticas mais profundas e capacidades avançadas de geração textual [26]. Nessa abordagem, o código-fonte é tratado como uma sequência de tokens, permitindo que o modelo aprenda mapeamentos entre a sequência de entrada (código) e a sequência de saída (sumários, JavaDocs e DocStrings). O mecanismo de atenção possibilita ao modelo focar em partes relevantes do código durante o processo

de geração, melhorando a fluência textual e a cobertura semântica em comparação com abordagens anteriores. Além da sumarização automática, os LLMs passaram a ser empregados em diversas atividades de engenharia de software, incluindo geração de documentação, explicação de código, refatoração, geração de testes e assistência ao desenvolvimento de software de forma cada vez mais contextualizada e semanticamente consistente [27]. Estudos recentes indicam que representações estruturais isoladas, como ASTs serializadas, apesar de preservarem parte significativa da estrutura sintática do programa, apresentam limitações na modelagem de informações semânticas mais profundas, incluindo relações entre identificadores, dependências de dados, contexto arquitetural e intenção do desenvolvedor. Como consequência, parte da expressividade semântica do software pode ser perdida durante o processo de representação, afetando diretamente a qualidade das tarefas de compreensão e sumarização automática de código [28]. Nesse contexto, abordagens híbridas passaram a incorporar múltiplas formas de representação estrutural e semântica, combinando tokens, ASTs, grafos de fluxo de

dados (DFG) e mecanismos de atenção profunda para melhorar a capacidade dos modelos em compreender relações contextuais e dependências presentes no software. Apesar da evolução recente dos modelos neurais e dosFoundation Modelsaplicados à sumarização automática de código, a literatura ainda evidencia limitações relevantes relacionadas à dependência de grandes volumes dedatasetsparalelos contendo pares código–documentação. Em geral, esses conjuntos de dados concentram-se em linguagens amplamente utilizadas e em repositóriosopen-sourceespecíficos, o que pode introduzir vieses de domínio, restringir a diversidade arquitetural dos exemplos e dificultar a capacidade de generalização dos modelos para sistemas proprietários, legados ou pouco documentados [29]. Estudos recentes reforçam essa limitação. Hu et al. (2025) observam que a avaliação de LLMs em engenharia de software permanece fortemente baseada em benchmarks públicos e conjuntos de dados experimentais, frequentemente construídos em ambientes controlados e com disponibilidade limitada de artefatos industriais reais. Complementarmente, Zhu et al. (2025) apontam desafios relacionados à representatividade e à generalização dos datasets de defeitos de software utilizados na literatura. Essas limitações evidenciam uma lacuna quanto à validação empírica de técnicas baseadas em LLMs em sistemas corporativos complexos e em operação contínua [30]. Para reduzir problemas relacionados à perda de contexto e inconsistências semânticas, abordagens recentes passaram a incorporar estratégias deRetrieval-Augmented Generation(RAG), combinando modelos generativos com mecanismos de recuperação de informações relevantes diretamente do repositório de código. Essas abordagens utilizam elementos estruturais e contextuais do sistema como classes, métodos, atributos, relacio-

namentos e anotações para fornecergroundingadicional ao modelo durante o processo de geração [31].Em paralelo, pesquisas recentes têm incorporado mecanismos de validação, comparação estrutural e análise estática ao processo de geração automática de documentação e modelos de software, buscando não apenas produzir descrições em linguagem natural, mas também verificar sua consistência em relação à estrutura real do código, às dependências internas e aos elementos recuperados do sistema [32]. Em síntese, a evolução desource code summarizationcaminha de técnicas puramente textuais para abordagens que integram estrutura de programa, recuperação de contexto e, mais recentemente, LLMs de propósito geral adaptados à engenharia de software.

2.2 LLMs, FerramentasEspecíficaseFoundationMo-

dels Com a consolidação recente da inteligência artificial generativa aplicada à engenharia de software, ferramentas baseadas emLarge Language Models(LLMs) passaram a ser incorporadas progressivamente aos fluxos de desenvolvimento como mecanismos de assistência cognitiva voltados à compreensão de código, geração de documentação, revisão de alterações, manutenção e evolução de sistemas de software [16]. Diferentemente de modelos tradicionais especializados em tarefas específicas,

os LLMs atuam como modelos de base (foundation models), capazes de serem adaptados a diferentes cenários por meio de técnicas comoprompting,few-shot learningefine-tuning. No contexto da engenharia de software, os LLMs têm sido empregados em diversas atividades relacionadas ao desenvolvimento e manutenção de sistemas, incluindo depuração, geração automática de testes, revisão de código, rastreabilidade, sumarização e produção automatizada de documentação técnica [33]. Em particular, a geração de documentação tem despertado crescente interesse da comunidade científica por seu potencial de apoiar a compreensão de sistemas complexos, reduzir o esforço cognitivo dos desenvolvedores e mitigar problemas associados à ausência ou desatualização de artefatos documentais [34]. Modelos especializados em código, comoCodeBERT, bem como arquiteturas mais recentes baseadas emTransformerseFoundation Models, têm obtido resultados expressivos em tarefas de compreensão de software esource code summarization[35]. Esses modelos exploram representações contextuais capazes de relacionar linguagem natural e código-fonte, permitindo gerar descrições mais consistentes e semanticamente alinhadas ao comportamento do software. Entretanto, estudos recentes indicam que pipelines baseados em LLMs requerem mecanismos complementares de validação automatizada e supervisão humana contínua, uma vez que métricas tradicionais de similaridade textual nem sempre refletem adequadamente

a qualidade semântica, a corretude técnica e a utilidade prática dos artefatos gerados automaticamente [13]. Nesse cenário, diferentes provedores de tecnologia passaram a disponibilizar LLMs integrados a plataformas corporativas, oferecendo recursos voltados à engenharia de software, governança e integração com repositórios privados. O Amazon Q [36] representa uma dessas soluções, fornecendo suporte a atividades como compreensão de código, navegação por repositórios, geração de documentação e assistência ao desenvolvimento em ambientes AWS. Nesta pesquisa, entretanto, o Amazon Q não constitui o objeto de investigação em si, mas sim a tecnologia utilizada para operacionalizar o estudo empírico sobre geração automatizada de documentação em sistemas legados. A literatura também destaca desafios recorrentes associados ao uso de LLMs para compreensão e documentação de software. Entre eles destacam-se: (i) a dificuldade de lidar com bibliotecas proprietárias, regras de negócio implícitas e dependências específicas de cada repositório; (ii) limitações de janela de contexto em sistemas extensos, exigindo mecanismos de recuperação e seleção de informações relevantes; e (iii) riscos de alucinação, nos quais o modelo produz descrições inconsistentes com o comportamento real do sistema [37]. Tais limitações têm motivado a integração de LLMs com técnicas complementares, incluindo análise estática, recuperação de contexto, grafos de dependência e mecanismos estruturados de avaliação, buscando aumentar a confiabilidade e a qualidade dos artefatos documentais gerados automaticamente.

2.3 Validação de Sumarizações e Avaliação da Docu-

mentação Gerada Embora métricas automáticas como BLEU, ROUGE e métricas tradicionais de similaridade textual sejam amplamente utilizadas na avaliação de sumarização de código, estudos recentes demonstram que tais abordagens apresentam limitações significativas para capturar utilidade prática, valor semântico e impacto cognitivo percebido por desenvolvedores durante atividades reais de compreensão de software [20]. Como consequência, um sumário pode apresentar alta similaridade textual com uma referência e, ainda assim, ser impreciso, incompleto ou pouco útil em cenários reais de desenvolvimento. Diante dessas limitações, a literatura recente tem enfatizado a necessidade de abordagensdeavaliaçãoquetranscendammétricaspuramentelexicaisebaseadasemsimilaridade textual, incorporando dimensões relacionadas à utilidade prática, qualidade semântica e apoio à compreensão de software em cenários reais de engenharia. Estudos recentes demonstram que mesmo abordagens mais sofisticadas baseadas em Large Language Models

(LLMs) utilizados como avaliadores (LLM-as-a-judge) ainda apresentam limitações relevantes quando comparadas a mecanismos objetivos de validação. Bavota et al. [38] observaram que LLMs podem apresentar correlação significativa com avaliações humanas em tarefas de sumarização de código, superando métricas tradicionais baseadas em sobreposição textual. Entretanto, evidências mais recentes indicam que a utilização de modelos de linguagem como juízes automáticos ainda requer cautela. Evidências recentes também sugerem cautela na utilização de Large Language Models como avaliadores automáticos de artefatos de software. Bavota e al. demonstraram que modelos especificamente ajustados para atuar como juízes de corretude de código alcançam apenas níveis moderados de concordância com oráculos baseados em execução de testes, indicando que avaliações automatizadas fundamentadas em LLMs ainda podem divergir de mecanismos objetivos de validação. Esses resultados reforçam a necessidade de abordagens híbridas de avaliação, combinandométricasautomáticas, validaçõesestruturaisejulgamentoshumanosespecializados. Dentro dessa realidade, estudos recentes têm explorado abordagens que integram análise estrutural do código, avaliação semântica, mecanismos automatizados de validação e julgamentos fundamentados na percepção de desenvolvedores e especialistas humanos [20]. As avaliações fundamentadas na percepção de desenvolvedores tornam-se particularmente relevantes, pois permitem investigar dimensões frequentemente pouco representadas por métricas automáticas tradicionais, como apoio à compreensão de software, clareza das explicações produzidas, relevância contextual e utilidade prática das sumarizações em cenários reais de manutenção e evolução de software [39]. Em ambientes reais de engenharia de software, a utilidade da documentação está fortemente relacionada à sua capacidade de apoiar atividades práticas de compreensão, manutenção e rastreabilidade do sistema, além de facilitar a integração de novos contribuidores e o compartilhamento de conhecimento entre membros da equipe [40]. Diante dessas limitações, a literatura recente tem enfatizado

a necessidade de
abordagensdeavaliaçãoquetranscendammétricaspuramentelexicaisebaseadasemsimilaridade textual, incorporando dimensões relacionadas à utilidade prática, qualidade semântica e apoio à compreensão de software em cenários reais de engenharia. Nesse contexto, estudos recentes têm explorado abordagens híbridas que combinam análise estrutural do código, avaliação semântica, mecanismos automatizados de validação e julgamentos fundamentados na percepção de desenvolvedores e especialistas humanos [20] Em ambientes dessa natureza, avaliações fundamentadas na percepção de desenvolvedores tornam-se particularmente relevantes, pois permitem investigar dimensões frequentemente pouco representadas por métricas automáticas tradicionais, como apoio à compreensão de software, clareza das explicações produzidas, relevância contextual e uti-

lidade prática das sumarizações em cenários reais de manutenção e evolução de software [39]. Em ambientes reais de engenharia de software, a utilidade da documentação está fortemente relacionada à sua capacidade de apoiar atividades práticas de compreensão, manutenção e rastreabilidade do sistema, além de facilitar a integração de novos contribuidores e o compartilhamento de conhecimento entre membros da equipe [40]. Considerando esse panorama, estudos empíricos recentes têm empregado abordagens qualitativas exploratórias para investigar como funcionalidades de software são documentadas e rastreadas em projetos colaborativos de grande porte. Puhlfurß, Montgomery e Maalej [40], por exemplo, realizaram uma análise qualitativa de 25 projetos populares hospedados no GitHub com o objetivo de compreender como o conhecimento sobre funcionalidades é documentado e relacionado ao código-fonte correspondente. Os resultados mostraram que a documentação encontra-se frequentemente distribuída entre diferentes artefatos, como arquivos README, wikis e recursos de websites, além de apresentar descrições pouco estruturadas e reduzido uso de mecanismos explícitos de rastreabilidade. Os autores também observaram grande heterogeneidade nas estratégias de documentação empregadas pelos projetos analisados, indicando a ausência de práticas amplamente consolidadas para organização e vinculação entre documentação e implementação. Como desdobramento, o estudo sugere a realização de investigações complementares envolvendo contribuidores dos projetos por meio de entrevistas e questionários, com o objetivo de compreender as motivações e percepções associadas às decisões de documentação. Outro aspecto relevante refere-se à necessidade de avaliar a documentação em contextos reais de uso. Parte significativa da literatura ainda fundamenta suas avaliações em datasets públicos e cenários experimentais distantes das condições reais da indústria, o que restringe a compreensão sobre a capacidade de adaptação e o desempenho dessas técnicas em sistemas corporativos complexos, altamente especializados e institucionalmente mantidos [41]. Em contraste, estudos realizados em contextos institucionais e operacionais permitem analisar como informações técnicas e artefatos de software são efetivamente utilizados no cotidiano das equipes, considerando restrições reais de contexto, conhecimento de domínio e

dinâmica organizacional [42], considerando fatores como pressão por prazos, heterogeneidade do código e dependência de conhecimento tácito. Dessa forma, a avaliação da documentação gerada por modelos de linguagem deve ser compreendida como um processo multidimensional, que envolve tanto aspectos técnicos quanto humanos. Enquanto análises estruturais podem contribuir para verificar a consistência e a aderência ao código, a percepção dos desenvolvedores é fundamental para avaliar a utilidade prática e a efetividade da documentação no contexto da engenharia de software.

Alinhado a essa perspectiva, este trabalho adota uma abordagem de avaliação qualitativa baseada em grupo focal, no qual desenvolvedores analisam a documentação gerada automaticamente em um cenário real de revisão de código. Essa estratégia permite observar não apenas a qualidade do artefato gerado, mas também sua aplicabilidade em atividades reais, contribuindo para uma avaliação mais robusta e aderente ao contexto industrial. Em síntese, a validação de sumarizações de código não pode depender exclusivamente de métricas automáticas, sendo necessário integrar diferentes formas de avaliação que considerem tanto a estrutura do código quanto a experiência dos desenvolvedores. Essa abordagem híbrida, com ênfase na avaliação qualitativa, fundamenta a metodologia adotada nesta pesquisa.

2.4 Redocumentação e sistemas legados

Redocumentação é o processo de reconstruir, atualizar ou complementar documentação a partir de artefatos existentes do software. Em sistemas legados, esse processo procura recuperar intenção, comportamento e relações que deixaram de estar disponíveis em documentos atualizados. Nesta pesquisa, o termo é usado em sentido restrito: reconstrução de JavaDocs a partir do código-fonte e de seu contexto estrutural, sem assumir que comentários gerados substituam documentação arquitetural ou conhecimento de domínio.

2.5 Arquitetura multiagente

Uma arquitetura multiagente distribui responsabilidades entre componentes autônomos especializados que cooperam para alcançar um objetivo comum. No pipeline estudado, agente designa um componente de software com uma responsabilidade delimitada e uma interface de entrada e saída, e não uma entidade humana. A decomposição permite separar inventário estrutural, priorização, geração, validação e integração, tornando decisões e falhas observáveis. O uso de agentes é justificado quando essas responsabilidades exigem políticas distintas e registros independentes; ele não implica que todos os componentes empreguem um modelo de linguagem.

Agente	Responsabilidade
Agente de	Executa o JavaParser, identifica alvos públicos e produz o

inventário	manifest.json.
Agente de priorização	Seleciona alvos segundo camada, complexidade, trivialidade e relevância funcional.
Agente de geração	Monta o contexto e solicita ao modelo um JavaDoc conforme idioma e template definidos.
Agente de validação	Verifica formato, aderência ao alvo, presença de elementos obrigatórios e ausência de alterações não documentais.
Agente de integração	Insere somente comentários aceitos, registra métricas e organiza os artefatos de saída.

2.6 JavaDoc como artefato avaliado

JavaDoc é o formato padronizado de comentários de documentação da plataforma Java. Um JavaDoc útil deve explicar propósito e comportamento, registrar parâmetros, retorno e exceções quando aplicáveis e acrescentar informação que não seja imediatamente evidente pela assinatura. Fluência textual não garante correção nem utilidade; por isso, a avaliação combina evidências estruturais e julgamento humano contextual.

2.7 Síntese dos trabalhos relacionados

Linha de trabalho	Ênfase	Diferença desta pesquisa
Sumarização baseada em regras e modelos neurais	Geração automática e métricas de similaridade	A pesquisa avalia JavaDocs em revisão real por mantenedores do sistema.
LLMs para documentação de código	Fluência, cobertura semântica e benchmarks públicos	O caso utiliza sistema público institucional, linha de base e artefatos de execução.
LLM como juiz	Avaliação automatizada	A validação automática é complementar; a decisão de utilidade permanece humana.
Documentação	Distribuição e	O estudo observa integração

em projetos colaborativos	rastreabilidade de artefatos	por Merge Request e custo cognitivo da revisão.

Capítulo 3

Método de pesquisa

3.1 Abordagem e objeto de estudo

A pesquisa adota abordagem qualitativa exploratória apoiada por evidências técnicas. O objeto de estudo é a documentação JavaDoc produzida e revisada no processo de redocumentação. O SARSB constitui o caso de aplicação, e a arquitetura multiagente é o instrumento tecnológico utilizado para produzir os artefatos. Essa distinção evita confundir sistema, ferramenta e fenômeno investigado.

3.2 Rastreabilidade da pesquisa

uestão	Foco	Dados	Análise	Produto
P	Qualidade, utilidade, confiabilidade e adoção de JavaDocs	Comentários no GitLab, grupo focal, artefatos e relatórios	Análise temática e estatística descritiva	Categorias, exemplos e condições de adoção
Q1	Componentes de maior valor	Tipo de alvo, classe, camada e avaliações	Comparação por tipo e interpretação contextual	Prioridade de geração
Q2	Problemas gerados	Excertos, comentários, falhas e diffs	Codificação temática e contagens verificáveis	Redundância, inconsistência e risco
Q3	Influência de escopo e entrega	Percepções sobre seleção e Merge Request	Análise temática	Requisitos para revisão e adoção

3.3 Linha de base documental

O inventário reproduzido identificou 1478 alvos públicos. Destes, 9 possuíam JavaDoc (0.61%) e 1469 não possuíam (99.39%). A distribuição foi de 521 classes, 866 métodos, 84 construtores e 7 enumerações. Esses valores são usados como baseline da versão analisada.

3.4 Arquitetura e fluxo operacional do pipeline

O fluxo começa com um arquivo ZIP contendo o projeto Java. O arquivo é armazenado temporariamente em Amazon S3 e um job é registrado e enviado ao AWS CodeBuild. O pipeline localiza a raiz do código, executa o JavaParser, produz o manifesto estrutural, aplica a política de seleção e monta um contexto por alvo. O contexto inclui tipo, caminho, nome qualificado, assinatura, intervalo de linhas, anotações e trecho de código. O agente de geração invoca o modelo por Amazon Bedrock; o agente de validação verifica a resposta; o agente de integração insere o JavaDoc aceito sem autorizar mudanças executáveis. Ao final, código documentado, manifest.json, javadoc_report.json e logs são armazenados e uma amostra é apresentada em Merge Request para revisão.

Os serviços S3, CodeBuild, Lambda, SQS, DynamoDB e EventBridge realizam armazenamento e orquestração. Eles não são variáveis empíricas. Os parâmetros de modelo, limite de alvos, idioma, template e política de seleção devem ser registrados em cada execução. Um controle de integridade compara a AST antes e depois e bloqueia diffs que alterem imports, assinaturas ou corpos de métodos. A versão do Estudo 01 não impediu integralmente alterações fora do escopo, razão pela qual esse controle é requisito explícito do refinamento.

3.5 Protocolo do Estudo 01

- Participantes: três desenvolvedores com experiência em desenvolvimento e revisão de código; um pesquisador atuou como facilitador.
- Artefatos: dez classes documentadas, código original e modificado, uma Merge Request, 15 comentários de revisão, logs, manifesto e relatório de geração.
- Sessão: aproximadamente uma hora, com introdução, contextualização, análise individual, discussão coletiva e encerramento.
- Perguntas: utilidade e clareza; valor por classe ou método; redundância; correção; idioma e consistência; esforço de revisão; confiança e adoção.
- Análise: seleção de 35 excertos, codificação próxima às falas, agrupamento em categorias interpretativas, busca de recorrências, divergências, casos positivos e sugestões.
- Critério de conclusão: cada interpretação deve ser sustentada por artefato, comentário ou excerto rastreável, sem pretensão de generalização estatística.

3.6 Amostragem

A amostra intencional compreendeu dez classes distribuídas entre DTOs, entidades, serviços, utilitários e componentes de relatórios. Os critérios foram diversidade arquitetural, ausência de documentação prévia, variação de complexidade, relevância funcional e possibilidade de revisão pelos participantes. A única Merge Request preservou um ambiente uniforme de comparação, mas elevou o custo cognitivo; por isso, o Estudo 02 deve organizar entregas menores por contexto ou camada.

3.7 Planejamento do Estudo 02

O Estudo 02 permanecerá concentrado em JavaDocs. Ele utilizará as categorias do Estudo 01 como dimensões a priori, comparará as versões do pipeline e manterá abertura para códigos emergentes. As medidas previstas são: proporção de alvos selecionados por camada; taxa de JavaDocs aceitos; incidência de redundância; conformidade de idioma e template; número de alterações não documentais; correções solicitadas; tempo e esforço percebido de revisão. Resultados somente serão apresentados após a coleta.

3.8 Ameaças à validade

Dimensão	Risco	Mitigação
Construto	Utilidade percebida pode não equivaler a ganho objetivo de manutenção.	Triangulação entre falas, comentários, exemplos e métricas de execução.
Interna	Conhecimento prévio, facilitação e formato da MR podem influenciar respostas.	Roteiro neutro, análise individual anterior à discussão e registro do procedimento.
Externa	Um sistema, três participantes e amostra intencional limitam generalização.	Descrição detalhada do contexto e replicação planejada no Estudo 02.
Confiabilidade	Codificação qualitativa por um pesquisador pode introduzir interpretação subjetiva.	Codebook explícito, trilha de decisões e revisão independente quando disponível.
Conclusão	Frequências pequenas não sustentam inferência estatística.	Uso descritivo das contagens e preservação das evidências qualitativas.

Capítulo 4

Estudo 01 avaliação preliminar de JavaDocs gerados por modelos de linguagem

4.1 Objetivo e questão

O estudo analisou como desenvolvedores percebem a qualidade e a utilidade de JavaDocs gerados automaticamente durante revisão de código. A questão foi operacionalizada pelas três subquestões apresentadas na matriz de rastreabilidade.

4.2 Execução e evidências técnicas

A execução reproduzida processou 1478 alvos. O pipeline registrou 1466 inserções, 9 alvos ignorados e 3 respostas rejeitadas na validação. Foram tentadas 1466 chamadas ao modelo, com 1463 respostas aceitas pelo fluxo e 0 falhas de chamada. Esses números caracterizam a execução e não demonstram, isoladamente, qualidade semântica.

4.3 Corpus e análise temática

O corpus reuniu dez classes, 15 comentários no GitLab e 35 excertos selecionados da transcrição do grupo focal. A planilha de intercâmbio ATLAS.ti disponível contém o codebook, porém não contém as citações codificadas. Assim, esta versão quantifica somente unidades verificáveis e não atribui frequências não recuperáveis às categorias. A exportação completa das citações deve ser preservada no pacote de replicação antes da análise final do Estudo 02.

4.4 Resultados e exemplos concretos

Categoria	Situação	Evidência	Interpretação
Utilidade	Método de serviço complexo	O JavaDoc ajuda quando explica regra ou fluxo que não é evidente.	Valor semântico proporcional à complexidade.
Redun	DTO ou	Comentário repete	Ruído e custo

dância	entidade simples	nome, atributos ou operação visível.	futuro de sincronização.
Padronização	Comentários em português e inglês	Variação terminológica entre artefatos semelhantes.	Necessidade de idioma e template únicos.
Risco técnico	Inclusão de import desnecessário	Mudança fora do escopo documental.	Revisão humana e bloqueio de diff executável.
Entrega	MR exclusivamente documental	Revisor precisa reconstruir contexto funcional que não acompanhou.	Entregas menores e contextualizadas.

4.4.1 Utilidade e valor semântico

"Em alguns métodos, ela ajuda bastante. Dependendo de ser um método complexo de entender, ela ajuda bastante, mas, em alguns casos, ela foi bem superficial."

A fala associa utilidade à complexidade e não apenas à fluência do texto.

4.4.2 Documentação seletiva

"Nos métodos do serviço é o que a gente mais usa. Documentação de classe, entidade e DTO, só se for mais detalhada."

A evidência sustenta priorização de serviços e uso condicionado em estruturas simples.

4.4.3 Sustentabilidade

"Pode ser que a documentação sinalize um texto e o desenvolvedor veja que o código não bate."

A divergência cria duas possíveis fontes de verdade e aumenta a dívida documental.

4.4.4 Risco e revisão humana

"Era uma ferramenta de documentação que importou pacote. Muita gente não usa IA de maneira supervisionada; pode dar problema."

A alteração indevida afeta integridade do código e confiança no pipeline.

4.4.5 Estratégia de entrega

“Eu vou ter muito mais trabalho do que facilidade, porque estou tendo que aprender e conhecer um negócio que não fui eu que fiz.”

O custo de revisão decorre da ausência de contexto funcional, não apenas do tamanho da MR.

4.5 Lições e rastreabilidade para o Estudo 02

Achado do Estudo 01	Mudança	Medida no Estudo 02
Redundância	Filtro de trivialidade e prompt orientado a valor semântico	Taxa de redundância e aceitação
Maior valor em serviços	Priorização por camada, complexidade e regra de negócio	Aceitação por tipo de componente
Alterações indevidas	Comparação AST e bloqueio de diffs não documentais	Número de alterações executáveis
Idioma e formato	Contrato de geração e template único	Conformidade ao padrão
Custo de revisão	Entregas menores por contexto	Tempo, correções e esforço percebido
Dependência do domínio	Revisão por mantenedores e registro do contexto	Concordância entre perfis de avaliador

Capítulo 5

Planejamento da continuidade

O Estudo 02 será executado somente após congelamento do protocolo, da versão do pipeline e dos critérios de análise. A coleta manterá JavaDocs como único tipo documental avaliado e registrará modelo, parâmetros, prompt, idioma, política de seleção, versão do código e hash dos artefatos.

Etapa	Produto verificável	Situação
Refinamento do pipeline	Versão etiquetada, controles de integridade e configuração registrada	Previsto
Piloto do protocolo	Instrumento revisado e teste de compreensão	Previsto
Execução comparativa	Saídas das versões 1 e 2 com hashes	Previsto
Coleta com desenvolvedores	Consentimentos, comentários, tempos e avaliações	Previsto
Análise	Matriz achado mudança efeito e codebook final	Previsto
Redação final	Resultados, discussão, limitações e conclusão	Previsto

Capítulo 6

Contribuições e impacto

A contribuição científica mais defensável é a cadeia de evidências que liga uma lacuna documental observada em sistema real, a geração automatizada de JavaDocs, a revisão por mantenedores, as categorias qualitativas e o refinamento mensurável do pipeline. Para a manutenção de software, o estudo fornece critérios para decidir onde documentar, como revisar e quais riscos controlar. Para a

academia, oferece protocolo e artefatos de replicação aplicáveis a avaliações de documentação gerada por LLMs.

O impacto social decorre do contexto público. Sistemas que apoiam regulação e políticas de saneamento dependem de continuidade operacional e transferência de conhecimento entre equipes. JavaDocs não resolvem isoladamente problemas arquiteturais ou institucionais, mas podem reduzir barreiras de compreensão, apoiar integração de novos profissionais e tornar decisões de manutenção mais rastreáveis. Esses benefícios são condicionados à correção, à revisão humana e à proteção dos dados e do código institucional.

Limitações

O Estudo 01 envolve um único sistema, três participantes, dez classes e uma única Merge Request. A amostragem foi intencional, a avaliação é predominantemente perceptiva e a codificação foi conduzida em contexto exploratório. A planilha ATLAS.ti disponível não preserva as citações codificadas, o que impede frequências por categoria nesta versão. O Estudo 02 deve ampliar a rastreabilidade, comparar versões e preservar o pacote empírico completo, sem reivindicar generalização estatística.

Referências

[1] Godfrey, Michael W. e Daniel M. German:The past, present, and future of software evolution. Em2008 Frontiers of Software Maintenance, páginas 129–138, 2008. 1

[2] Khadka, Ravi, Belfrit V. Batlajery, Amir Saeidi, Slinger Jansen e Jurriaan Hage:How do professionals perceive legacy systems and software modernization?Em Jalote, Pankaj, Lionel C. Briand e André van der Hoek (editores):36th International Con- ference on Software Engineering, ICSE '14, Hyderabad, India - May 31 - June 07, 2014, páginas 36–47. ACM, 2014.https://doi.org/10.1145/2568225.2568318. 1

[3] Le, Duc Minh, Pooyan Behnamghader, Joshua Garcia, Daniel Link, Arman Shah- bazianeNenadMedvidovic:An empirical study of architectural change in open-source software systems. Em Penta, Massimiliano Di, Martin Pinzger e Romain Robbes (edi- tores):12th IEEE/ACM Working Conference on Mining Software Repositories, MSR 2015, Florence, Italy, May 16-17, 2015, páginas 235–245. IEEE Computer Society, 2015.https://doi.org/10.1109/MSR.2015.29. 1

[4] Shahbazian, Arman, Youn Kyu Lee, Duc Minh Le, Yuriy Brun e Nenad Medvi- dovic:Recovering architectural design decisions. EmIEEE International Conference on Software Architecture, ICSA 2018, Seattle, WA, USA, April 30 - May 4, 2018, páginas 95–104. IEEE Computer Society, 2018.https://doi.org/10.1109/ICSA. 2018.00019. 1

[5] Hoff, Adrian, Michael Nieke e Christoph Seidl:Towards immersive software ar- chaeology: regaining legacy systems' design knowledge via interactive exploration in virtual reality. Em Spinellis, Diomidis, Georgios Gousios, Marsha Chechik e Mas- similiano Di Penta (editores):ESEC/FSE '21: 29th ACM Joint European Soft- ware Engineering Conference and Symposium on the Foundations of Software En- gineering, Athens, Greece, August 23-28, 2021, páginas 1455–1458. ACM, 2021. https://doi.org/10.1145/3468264.3473128. 1

[6] Harman, Mark:Why source code analysis and manipulation will always be important. EmTenth IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2010, Timisoara, Romania, 12-13 September 2010, páginas 7–19. IEEE Computer Society, 2010.https://doi.org/10.1109/SCAM.2010.28. 1

[7] Visconti, Marcello e Curtis R. Cook:An overview of industrial software documenta- tion practice. Em22nd International Conference of the Chilean Computer Science Society (SCCC 2002), 6-8 November 2002, Copiapo, Chile, páginas 179–186. IEEE Computer Society, 2002.https://doi.org/10.1109/SCCC.2002.1173192. 2

[8] Albuquerque, Danyllo, Everton Guimarães, Graziela Tonin, Pilar Rodríguez, Mirko Perkusich, Hyggo O. Almeida, Angelo Perkusich e Ferdinandy Chagas:Managing technical debt using intelligent techniques - A systematic mapping study. IEEE Trans. Software Eng., 49(4):2202–2220, 2023.https://doi.org/10.1109/TSE. 2022.3214764. 2

[9] Siebra, Clauirton de, Antonio L. O. Cavalcanti, Fabio Q. B. da Silva, André L. M. Santos e Tatiana B. Gouveia:Applying metrics to identify and monitor technical debt items during software evolution. Em25th IEEE International Symposium on Software Reliability Engineering Workshops, ISSRE Workshops, Naples, Italy, November 3-6, 2014, páginas 92–95. IEEE Computer Society, 2014.https://doi.org/10.1109/ ISSREW.2014.59. 2

[10] Dearstyne, Katherine R., Alberto D. Rodriguez e Jane Cleland-Huang:Supporting software maintenance with dynamically generated document hierarchies. EmIEEE International Conference on Software Maintenance and Evolution, ICSME 2024, Flagstaff, AZ, USA, October 6-11, 2024, páginas 426–437. IEEE, 2024.https: //doi.org/10.1109/ICSME58944.2024.00046. 2

[11] Wu, Yang, YaoWan, ZhaoyangChu, WentingZhao, YeLiu, HongyuZhang, Xuanhua Shi, Hai Jin e Philip S. Yu:Can large language models serve as evaluators for code summarization?IEEE Trans. Software Eng., 51(12):3205–3217, 2025.https:// doi.org/10.1109/TSE.2025.3595283. 2

[12] Mayer, Richard, Michael Moser e Verena Geist:Leveraging and evaluating automatic code summarization for JPA program comprehension. Em Zhang, Tao, Xin Xia e Nicole Novielli (editores):IEEE International Conference on Software Analysis, Evo- lution and Reengineering, SANER 2023, Taipa, Macao, March 21-24, 2023, páginas 768–772. IEEE, 2023.https://doi.org/10.1109/SANER56733.2023.00088. 2, 8

[13] Sun, Weisong, Yun Miao, Yuekang Li, Hongyu Zhang 0002, Chunrong Fang, Yi Liu 0069, Gelei Deng, Yang Liu 0003 e Zhenyu Chen 0001:Source code summarization in the era of large language models. Em47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025, pá- ginas 1882–1894. IEEE, 2025.https://doi.org/10.1109/ICSE55347.2025.00034. 3, 11

[14] Chen, Yuxuan, Mingwei Liu, Guangsheng Ou, Anji Li, Dekun Dai, Yanlin Wang e Zibin Zheng:Are decoder-only large language models the silver bullet for code search? IEEE Trans. Software Eng., 52(4):1215–1233, 2026.https://doi.org/10.1109/ TSE.2026.3657353. 3

[15] Grees, Yousab, Polina Iaremchuk, Ramtin Ehsani, Esteban Parra, Preetha Chat- terjee e Sonia Haiduc:Ai-powered commit explorer (APCE). EmIEEE Interna- tional Conference on Software

Maintenance and Evolution, ICSME 2025, Auck- land, New Zealand, September 7-12, 2025, páginas 915–919. IEEE, 2025.https: //doi.org/10.1109/ICSME64153.2025.00106. 3

[16] Pilone, Arthur:Ensuring code integrity in the era of ai-assisted software development. EmIEEE International Conference on Software Maintenance and Evolution, ICSME 2025, Auckland, New Zealand, September 7-12, 2025, páginas 870–872. IEEE, 2025. https://doi.org/10.1109/ICSME64153.2025.00095. 3, 10

[17] Li, Yikun, Mohamed Soliman, Paris Avgeriou e Maarten van Ittersum:Debtviz: A tool for identifying, measuring, visualizing, and monitoring self-admitted technical debt. EmIEEE International Conference on Software Maintenance and Evolution, ICSME 2023, Bogotá, Colombia, October 1-6, 2023, páginas 558–562. IEEE, 2023. https://doi.org/10.1109/ICSME58846.2023.00072. 3

[18] Siala, Hanan Abdulwahab e Kevin Lano:Towards using llms in the reverse engi- neering of software systems to object constraint language. EmIEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2025, Mon- treal, QC, Canada, March 4-7, 2025, páginas 1–6. IEEE, 2025.https://doi.org/ 10.1109/SANER64311.2025.00096. 5

[19] Zhu, Hao-Nan, Robert M. Furth, Michael Pradel e Cindy Rubio-González:From bugs to benchmarks: A comprehensive survey of software defect datasets. ACM Comput. Surv., 58(10):249:1–249:39, 2026.https://doi.org/10.1145/3797033. 5

[20] Kruse, Hans-Alexander, Tim Puhlfürß e Walid Maalej:Can developers prompt? A controlled experiment for code documentation generation [replication package] (ver- sion 1.4.0).https://doi.org/10.5281/zenodo.16413762, julho 2025.https: //doi.org/10.5281/zenodo.16413762, Accessed on YYYY-MM-DD. 5, 8, 11, 12

[21] Girjoaba, Andrei V. e Andrea Capiluppi:Refactoring legacy code using cleaning up cycles: An experience report. EmIEEE International Conference on Software Main- tenance and Evolution, ICSME 2024, Flagstaff, AZ, USA, October 6-11, 2024, pá- ginas 753–764. IEEE, 2024.https://doi.org/10.1109/ICSME58944.2024.00079.

[22] Lee, Noah, Rui Abreu 0001, Mehmet Yatbaz, Hang Qu e Nachiappan Nagappan: The engineering implications of code maintenance in practice. EmIEEE Interna- tional Conference on Software Maintenance and Evolution, ICSME 2022, Limassol, Cyprus, October 3-7, 2022, páginas 568–577. IEEE, 2022.https://doi.org/10. 1109/ICSME55016.2022.00078. 7

[23] Moran, Kevin, Ali Yachnes, George Purnell, Junayed Mahmud, Michele Tufano, Car- los Bernal-Cárdenas, Denys Poshyvanyk e Zach H'Doubler:An empirical investiga- tion into the use of image

captioning for automated software documentation. CoRR, abs/2301.01224, 2023.https://doi.org/10.48550/arXiv.2301.01224. 8

[24] 0001, Yanlin Wang, Yanxian Huang, Daya Guo, Hongyu Zhang 0002 e Zibin Zheng: Sparsecoder: Identifier-aware sparse transformer for file- level code summarization. EmIEEE International Conference on Software Analysis, Evolution and Reengineer- ing, SANER 2024, Rovaniemi, Finland, March 12-15, 2024, páginas 614–625. IEEE, 2024.https://doi.org/10.1109/SANER60148.2024.00068. 8

[25] Gong, Zi, Cuiyun Gao, Yasheng Wang, Wenchao Gu, Yun Peng e Zenglin Xu: Source code summarization with structural relative position guided transformer. Em IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2022, Honolulu, HI, USA, March 15-18, 2022, páginas 13–24. IEEE, 2022. https://doi.org/10.1109/SANER53432.2022.00013. 8

[26] Sridhara, Giriprasad, Sujoy Roychowdhury, Sumit Soman, Ranjani Hosakere Giree- sha e Ricardo Britto 0001:Icing on the cake: Automatic code summarization at ericsson. EmIEEE International Conference on Software Maintenance and Evolu- tion, ICSME 2024, Flagstaff, AZ, USA, October 6-11, 2024, páginas 689–700. IEEE, 2024.https://doi.org/10.1109/ICSME58944.2024.00073. 8

[27] 0001, Jian Gu, Pasquale Salza e Harald C. Gall:Assemble foundation models for au- tomatic code summarization. EmIEEE International Conference on Software Anal- ysis, Evolution and Reengineering, SANER 2022, Honolulu, HI, USA, March 15-18, 2022, páginas 935–946. IEEE, 2022.https://doi.org/10.1109/SANER53432.2022. 00112. 9

[28] Cai, Bo, Yaoxiang Yu e Yi Hu:Cssam: Code search via attention matching of code semantics and structures. EmIEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2023, Taipa, Macao, March 21-24, 2023, pá- ginas 402–413. IEEE, 2023.https://doi.org/10.1109/SANER56733.2023.00045.

[29] Xia, Meng, Shradha Maharjan e Myoungkyu Song:Automated code summarization by training large language models with crowdsourced knowledge. Em23rd IEEE/ACIS International Conference on Software Engineering Research, Management and Ap- plications, SERA 2025, Las Vegas, NV, USA, May 29-31, 2025, páginas 126–133. IEEE, 2025.https://doi.org/10.1109/SERA65747.2025.11154515. 9

[30] Takerngsaksiri, Wannita, Chakkrit Tantithamthavorn, Michael Fu, Jirat Pasuksmit, Kun Chen e Ming Wu:Code readability in the age of large language models: An industrial case study from atlassian. EmIEEE International Conference on Software Maintenance and Evolution, ICSME 2025, Auckland,

New Zealand, September 7-12, 2025, páginas 732–742. IEEE, 2025.https://doi.org/10.1109/ICSME64153.2025. 00073. 9

[31] Boronat, Artur e Jawad Mustafa:Mdre-llm: A tool for analyzing and applying llms in software reverse engineering. EmIEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2025, Montreal, QC, Canada, March 4-7, 2025, páginas 850–854. IEEE, 2025.https://doi.org/10.1109/SANER64311. 2025.00090. 10

[32] Shang, Xiuwei, Shaoyin Cheng, Guoqiang Chen, Yanming Zhang, Li Hu, Xiao Yu, Gangyang Li, Weiming Zhang 0001 e Nenghai Yu:How far have we gone in binary code understanding using large language models. EmIEEE International Conference on Software Maintenance and Evolution, ICSME 2024, Flagstaff, AZ, USA, October 6-11, 2024, páginas 1–12. IEEE, 2024.https://doi.org/10.1109/ICSME58944. 2024.00012. 10

[33] Khati, Dipin:Trustworthiness of large language models for code. Em47th IEEE/ACM International Conference on Software Engineering, ICSE 2025 - Companion Proceed- ings, Ottawa, ON, Canada, April 27 - May 3, 2025, páginas 208–210. IEEE, 2025. https://doi.org/10.1109/ICSE-Companion66252.2025.00063. 10

[34] 0094, Hao Li, Cor Paul Bezemer e Ahmed E. Hassan:Software engineering and foundation models: Insights from industry blogs using a jury of foundation models, 2025.https://doi.org/10.5281/zenodo.14873552. 10

[35] Dipongkor, Atish Kumar:Towards interpreting the behavior of large language mod- els on software engineering tasks. EmProceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2024, Lisbon, Portugal, April 14-20, 2024, páginas 255–257. ACM, 2024. https://doi.org/10.1145/3639478.3639798. 10

[36] Amazon Web Services:Amazon q developer, 2026.https://aws.amazon.com/pt/q/, acesso em 2026-05-16. 11

[37] Guan, Zhanming, Junlin Liu, Jierui Liu, Chao Peng, Dexin Liu, Ningyuan Sun, Bo Jiang, Wenchao Li, Jie Liu e Zhu Han:Contextmodule: Improving code completion via repository-level contextual information. CoRR, abs/2412.08063, 2024.https: //doi.org/10.48550/arXiv.2412.08063. 11

[38] Crupi, Giuseppe, Rosalia Tufano e Gabriele Bavota:Improving code generation via small language model-as-a-judge. CoRR, abs/2602.11911, 2026.https://doi.org/ 10.48550/arXiv.2602.11911. 12

[39] 0016, Abhishek Kumar, Partha Pratim Das 0001 e Partha Pratim Chakrabarti: Summarize me: The future of issue thread interpretation. EmIEEE International Conference on Software Maintenance and

Evolution, ICSME 2023, Bogotá, Colom- bia, October 1-6, 2023, páginas 341–345. IEEE, 2023.https://doi.org/10.1109/ ICSME58846.2023.00042. 12, 13

[40] Puhlfürß, Tim, Lloyd Montgomery e Walid Maalej:An exploratory study of doc- umentation strategies for product features in popular github projects [replication package] (version 4).https://doi.org/10.5281/zenodo.16413716, julho 2025. https://doi.org/10.5281/zenodo.16413716, Accessed on YYYY-MM-DD. 12,

[41] Bashir, Sarmad:Towards ai-centric requirements engineering for industrial systems. EmProceedings of the 2024 IEEE/ACM 46th International Conference on Soft- ware Engineering: Companion Proceedings, ICSE Companion 2024, Lisbon, Por- tugal, April 14-20, 2024, páginas 242–246. ACM, 2024.https://doi.org/10.1145/ 3639478.3639811. 13

[42] Ahmed, Toufique, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann 0001, Xuchao Zhang e Saravan Rajmohan:Recommending root-cause and mitigation steps for cloud incidents using large language models. Em45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14- 20, 2023, páginas 1737–1749. IEEE, 2023.https://doi.org/10.1109/ICSE48619. 2023.00149. 13

[43] Carvalho Souza, Murillo Edson de:Amazon q industry case study: Automated doc- umentation pipeline for legacy java systems.https://github.com/murilloed/ amazon-q-industry-case-study/tree/main, 2026. GitHub repository containing the AWS-based documentation pipeline, focus group methodology, experimental ar- tifacts, qualitative and quantitative analyses, and implementation details accessed in 2026. 21

[44] ATLAS.ti Scientific Software Development GmbH:ATLAS.ti Web: Software de análise qualitativa de dados, 2026.https://web.atlasti.com/, Acesso em: 20 jul. 2026. 23

[45] ATLAS.ti Scientific Software Development GmbH:ATLAS.ti: Software para análise temática do grupo focal 1, 2026.https://atlasti.com/, Acesso em: 12 ago. 2026. 24, 25, 26, 27